

# Python 3.0 and Object Oriented Programming

Presented by Eng. Meriam Gaied  
October 2026

# About Python 3.0

Python is a **high-level, interpreted, general-purpose programming language** known for its:

- Simple, readable syntax.
- Versatility (used in web development, data science, automation, AI, and more).
- Large standard library and strong community support.
- Interpreted means that it does not need to compile before running.

# Object Oriented Programming

OOP is a programming paradigm based on the concept of “**objects**”, which can contain:

- **Data** (attributes/properties)
- **Code** (methods/functions)

It helps organize code to model real-world things more naturally and promotes reusability, modularity, and easier maintenance.

# Concept Description

## **Class**

A blueprint for creating objects (defines properties and methods).

## **Object**

An instance of a class.

## **Encapsulation**

Bundling data and methods inside a class, hiding internal details.

## **Inheritance**

A class can inherit properties and methods from another class.

# Concept Description

## **Polymorphism**

Different classes can define methods with the same name

## **Abstraction**

Hiding complex implementation details and showing only necessary parts  
Using abstract base classes or interfaces

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal): # Inheritance
    def speak(self):
        print(f"{self.name} barks")

# Creating objects
animal = Animal("Generic Animal")
dog = Dog("Buddy")

animal.speak() # Output: Generic Animal makes a sound
dog.speak()    # Output: Buddy barks
```

# Types of Encapsulation

**Public Attributes:** accessible from anywhere.

attribute

**Protected Attributes:** accessible within the class and its subclasses.

`_attribute`

**Private Attributes:** accessible only within the class.

`__attribute`

## What is self in Python Classes?

- **self** represents the **instance of the class**.
- It is how you access **attributes** and **methods** inside the class from that particular object.
- In Python, it is **explicitly** passed as the first parameter to instance methods.

```
class Person:
    def __init__(self, name, age):
        self.name = name # Assign instance variable 'name'
        self.age = age    # Assign instance variable 'age'

    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

# Create an object of Person
p = Person("Alice", 30)
p.greet() # Output: Hello, my name is Alice and I am 30 years old.
```

# HAPPY NUMBERS PROBLEM

A happy number is a number defined by the following process:

Starting with any positive integer, replace the number by the sum of the squares of its digits.

Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.

Those numbers for which this process ends in 1 are happy.

Return true if  $n$  is a happy number, and false if not.

# Solution

```
def isHappy(self, n: int) -> bool:
    if n == 1 or n == 7: return True
    elif n < 10: return False
    else:
        result = 0
        for i in str(n):
            result = result + (int(i))**2
        return self.isHappy(result)
```

## Resources for Practicing

<https://leetcode.com/>

<https://algomap.io/roadmap>

<https://docs.python.org/3.13/>

<https://docs.python.org/3.13/library/index.html>

<https://docs.python.org/3/tutorial/classes.html>